The Complete Guide To Scrapy Spidermon, Start Monitoring in 5 Minutes!

If anyone has done a lot of web scraping, the one thing they always know is that your scrapers always break and degrade overtime.

Web scraping isn't like other software applications, where for the most part you control all the variables. In web scraping, you are writing scrapers that are trying to extract data from moving targets.

Websites can:

- Change the HTML structure of their pages.
- Implement new anti-bot countermeasures.
- Block whole ranges of IPs from accessing their site.

All of which can degrade or completely break your scrapers. Because of this, it is vital that you have a robust monitoring and alerting setup in place for your web scrapers so you can react immediately when your spiders eventually begin the brake.

In this guide, we're going to walk you through Spidermon, a Scrapy extension that is designed to make monitoring your scrapers easier and more effective.

1. What is Spidermon?
2. Integrating Spidermon
3. Spidermon Monitors
4. Spidermon MonitorSuites
5. Spidermon Actions
6. Item Validation
7. End-to-End Spidermon Example + Code

For more scraping monitoring solutions, then be sure to check out the full list of Scrapy monitoring options here, including ScrapeOps, the purpose built job monitoring & scheduling tool for web scraping. Live Demo Here

What is Spidermon?

Spidermon is a Scrapy extension to build monitors for Scrapy spiders. Built by the same developers that develop and maintain Scrapy, Spidermon is a highly versatile and customisable monitoring framework for Scrapy which greatly expands the default stats collection and logging functionality within Scrapy.

Spidermon allows you to create custom monitors that will:

- Monitor your scrapers with template & custom monitors.
- Validate the data being scraped from each page.
- Notify you with the results of those checks.

Spidermon is highly customisable, so if you can track a stat then you will be able to create a Spidermon monitor to monitor it in real-time.

Spidermon is centered around **Monitors**, **MonitorSuites**, **Validators** and **Actions**, which are then used to monitor your scraping jobs and alert you if any tests are failed.

Integrating Spidermon

Getting setup with Spidermon is straight forward, but you do need to manually setup your monitors after installing the Spidermon extension.

To get started you need to install the Python package:

pip install spidermon

Then add 2 lines to your `settings.py` file:

```
## settings.py

## Enable Spidermon
SPIDERMON_ENABLED = True
```

```
## Add In The Spidermon Extension
EXTENSIONS = {
'spidermon.contrib.scrapy.extensions.Spidermon': 500,
}
```

From here, you need to define your **Monitors**, **Validators** and **Actions**, then schedule them to run with your **MonitorSuites**. We will go through each of these in this guide.

Spidermon Monitors

The **Monitor** is the core piece of Spidermon. Inherited from unittest, a monitor is a series of Unit Tests you define that allows you to test the scraping stats of your job versus predefined thresholds you have defined.

Basic Monitors

Out of the box, Spidermon has a number of basic monitors built in which you just need to enable and configure in your projects/spiders settings to activate for your jobs.

Monitors	Description
ItemCountMonitor	Check if spider extracted the minimum number of items threshold.
ItemValidationMonitor	Check for item validation errors if item validation pipelines are enabled.
FieldCoverageMonitor	Check if field coverage rules are met.
ErrorCountMonitor	Check the number of errors versus a threshold.
WarningCountMonitor	Check the number of warnings versus a threshold.

Monitors	Description
FinishReasonMonitor	Check if a job finished for an expected finish reason.
RetryCountMonitor	Check if any requests have reached the maximum amount of retries and the crawler had to drop those requests.
DownloaderExceptionMonitor	Check the amount of downloader exceptions (timeouts, rejected connections, etc.).
SuccessfulRequestsMonitor	Check the total number of successful requests made.
TotalRequestsMonitor	Check the total number of requests made.

To use any of these monitors you will need to define the thresholds for each of them in your `settings.py` file or your spiders custom settings.

Custom Monitors

With Spidermon you can also create your own custom monitors that can do just about anything. They can work with any type of stat that is being tracked:

- ✅ Requests
- ✅ Responses
- ✅ Pages Scraped
- ✅ Items Scraped
- ✅ Item Field Coverage
- ✅ Runtimes
- ✅ Errors & Warnings
- ✅ Bandwidth
- ✅ HTTP Response Codes
- ✅ Retries
- ✅ Custom Stats

Basically, you can create a monitor to verify any stat that appears in the Scrapy

stats (either the default stats, or custom stats you configure your spider to insert).

Here is a example of a simple monitor that will check the number of items scraped versus a minimum threshold.

my_project/monitors.py

from spidermon import Monitor, monitors

@monitors.name('Item count') class CustomItemCountMonitor(Monitor):

@monitors.name('Minimum number of items') def test_minimum_number_of_items(self): item_extracted = getattr(self.data.stats, 'item_scraped_count', 0) minimum_threshold = 10

msg = 'Extracted less than items'.format(minimum_threshold) self.assertTrue(item_extracted >= minimum_threshold, msg=msg)

To run a **Monitor**, they need to be included in a **MonitorSuite**.

Spidermon MonitorSuites

A **MonitorSuite** is how you activate your **Monitors**. They tell Spidermon when you would like your monitors run and what actions should Spidermon take if your scrape passes/fails any of your health checks.

There are three built in types of monitors within Spidermon:

MonitorSuites	Description
SPIDERMON_SPIDER_OPEN_MONITORS	Runs monitors when Spider starts running.
SPIDERMON_SPIDER_CLOSE_MONITORS	Runs monitors when Spider has finished scraping.

MonitorSuites	Description
SPIDERMON_PERIODIC_MONITORS	Runs monitors are periodic intervals that you can define.

Within these **MonitorSuites** you can specify which actions should be taken after the **Monitors** have been executed.

To create a **MonitorSuite**, simply create a new **MonitorSuite** class, and define which monitors you want to run and what actions should be taken afterwards:

tutorial/monitors.py

from spidermon.core.suites import MonitorSuite

class SpiderCloseMonitorSuite(MonitorSuite): monitors = [CustomItemCountMonitor, ## defined above]

monitors_finished_actions = [

actions to execute when suite finishes its execution

]

monitors_failed_actions = [

actions to execute when suite finishes its execution with a failed monitor

]

Then add that **MonitorSuite** to the `SPIDERMON_SPIDER_CLOSE_MONITORS` tuple in your `settings.py` file.

##settings.py SPIDERMON_SPIDER_CLOSE_MONITORS = ('tutorial.monitors.SpiderCloseMonitorSuite',)

Now Spidermon will run this **MonitorSuite** at the end of every job.

Spidermon Actions

The final piece of your **MonitorSuite** are **Actions**, which define what happens after a set of monitors has been run.

Spidermon has pre-built **Action** templates already included, but you can easily create your own custom **Actions**.

Here is a list of the pre-built **Action** templates:

Actions	Description
Email	Send alerts or job reports to you and your team.
Slack	Send slack notifications to any channel.
Telegram	Send alerts or reports to any Telegram channel.
Job Tags	Set tags on your jobs when using Scrapy Cloud.
File Report	Create and save a HTML report locally.
S3 Report	Create and save a HTML report to a S3 bucket.
Sentry	Send custom messages to Sentry.

How example to get Slack notifications when a job fails one of your monitors, you can use the pre-built **SendSlackMessageSpiderFinished** action by adding your Slack details to your `settings.py` file:

```
##settings.py
SPIDERMON_SLACK_SENDER_TOKEN = '<SLACK_SENDER_TOKEN>'
```

```
    SPIDERMON_SLACK_SENDER_NAME = '<SLACK_SENDER_NAME>'
    SPIDERMON_SLACK_RECIPIENTS = ['@yourself', '#yourprojectchannel']

Then including **SendSlackMessageSpiderFinished** in your
**MonitorSuite**:

    ## tutorial/monitors.py
    from spidermon.core.suites import MonitorSuite
    from spidermon.contrib.actions.slack.notifiers import
SendSlackMessageSpiderFinished

    class SpiderCloseMonitorSuite(MonitorSuite):
        monitors = [
            CustomItemCountMonitor,
        ]

        monitors_failed_actions = [
            SendSlackMessageSpiderFinished,
        ]
```

Item Validation

One really powerful feature of Spidermon is its support for Item validation. Using schematics or JSON Schema, you can define custom unit tests on fields of each Item.

For example, we can have Spidermon test every product item we scrape has a valid product url, has a price that is a number and doesn't include any currency signs or special characters, etc.

Here is an example product item validator:

```
    ## validators.py
    from schematics.models import Model
    from schematics.types import URLType, StringType, ListType

    class ProductItem(Model):
        url = URLType(required=True)
        name = StringType(required=True)
        price = DecimalType(required=True)
        features = ListType(StringType)
```

Cannot get this to work,
Refer spidermon docs
The only method shown there uses jsonschema

```
        image_url = URLType()
```

Which can be enabled in your spider by activating Spidermons **ItemValidationPipeline** and telling Spidermon to use the **ProductItem** validator class we just created in your projects `settings.py` file.

```
    # settings.py
    ITEM_PIPELINES = {
        'spidermon.contrib.scrapy.pipelines.ItemValidationPipeline':
800,
    }

    SPIDERMON_VALIDATION_MODELS = (
        'tutorial.validators.ProductItem',
    )
```

This vaidator will then append new stats to your Scrapy stats which you can then use in your **Monitors**.

```
    ## log file
    ...
    'spidermon/validation/fields': 400,
    'spidermon/validation/items': 100,
    'spidermon/validation/validators': 1,
    'spidermon/validation/validators/item/schematics': True,
    [scrapy.core.engine] INFO: Spider closed (finished)
```

End-to-End Spidermon Example

Now, we're going to run through a full Spidermon example so that you can see how to setup your own monitoring suite.

The full code from this example is available on Github here.

Scrapy Project

First things first, we need a Scrapy project, a spider and a website to scrape. In this case books.toscrape.com.

scrapy startproject spidermon_demo scrapy genspider bookspider books.toscrape.com

Next we need to create a Scrapy Item for the data we want to scrape:

items.py

import scrapy

class BookItem(scrapy.Item): url = scrapy.Field() title = scrapy.Field() price = scrapy.Field()

Finally we need to write the spider code:

spiders/bookspider.py

import scrapy from spidermon_demo.items import BookItem

class BookSpider(scrapy.Spider): name = 'bookspider' start_urls = ["http://books.toscrape.com"]

def parse(self, response):

for article in response.css('article.product_pod'): book_item = BookItem(url = article.css("h3 > a::attr(href)").get(), title = article.css("h3 > a::attr(title)").extract_first(), price = article.css(".price_color::text").extract_first(),) yield book_item

next_page_url = response.css("li.next > a::attr(href)").get() if next_page_url: yield response.follow(url=next_page_url, callback=self.parse)

By now, you should have a working spider that will scrape every page of books.toscrape.com. Next we integrate Spidermon.

Integrate Spidermon

To install Spidermon just install the Python package:

pip install spidermon

Then add 2 lines to your `settings.py` file:

```
## settings.py

## Enable Spidermon
SPIDERMON_ENABLED = True

## Add In The Spidermon Extension
EXTENSIONS = {
'spidermon.contrib.scrapy.extensions.Spidermon': 500,
}
```

Next, we will define our validator for our BookItem model in a new `validators.py` file:

validators.py

from schematics.models import Model from schematics.types import URLType, StringType, ListType

class BookItem(Model): url = URLType(required=True) title = StringType(required=True) price = StringType(required=True)

Then enable this validator in our `settings.py` file:

```
## settings.py

ITEM_PIPELINES = {
    'spidermon.contrib.scrapy.pipelines.ItemValidationPipeline':
800,
}

SPIDERMON_VALIDATION_MODELS = (
    'spidermon_demo.validators.BookItem',
)
```

At this point, when you run your spider Spidermon will validate every item being scraped and update the Scrapy Stats with the results:

Scrapy Stats Output

(...) 'spidermon/validation/fields': 3000, 'spidermon/validation/fields/errors': 1000, 'spidermon/validation/fields/errors/invalid_url': 1000, 'spidermon/validation/fields/

errors/invalid_url/url': 1000, 'spidermon/validation/items': 1000, 'spidermon/validation/items/errors': 1000, 'spidermon/validation/validators': 1, 'spidermon/validation/validators/item/schematics': True,

We can see from these stats here, that the **url** field of our **BookItem** is failing all the validation checks. When digging deeper we will find that the reason is that scraped urls are relative urls `catalogue/a-light-in-the-attic_1000/index.html`, not absolute urls.

Create Our Monitors

Next, up we want to create **Monitors** that will conduct the unit tests when activated. In this example we're going to create two monitors in our `monitors.py` file.

Monitor 1: Item Count Monitor

This monitor will validate that our spider has scraped a set number of items.

monitors.py

@monitors.name('Item count') class ItemCountMonitor(Monitor):

@monitors.name('Minimum number of items') def test_minimum_number_of_items(self): item_extracted = getattr(self.data.stats, 'item_scraped_count', 0) minimum_threshold = 200

msg = 'Extracted less than items'.format(minimum_threshold) self.assertTrue(item_extracted >= minimum_threshold, msg=msg)

Monitor 2: Item Validation Monitor

This monitor will check the stats from Item validator to make sure we have no item validation errors.

monitors.py

@monitors.name('Item validation') class ItemValidationMonitor(Monitor, StatsMonitorMixin): "StatsMonitorMixin" is not defined, did not need it either

@monitors.name('No item validation errors') def test_no_item_validation_errors(self): validation_errors = getattr(self.stats, 'spidermon/validation/fields/errors', 0) self.assertEqual(validation_errors, 0,

msg='Found validation errors in fields'.format(validation_errors))

Create Monitor Suites

For this example, we're going to run two **MonitorSuites**. One at the end of the job, and another that runs every 5 seconds (for demo purposes).

MonitorSuite 1: Spider Close

Here, we're going to add both of our monitors (**ItemCountMonitor**,**ItemValidationMonitor**) to the monitor suite as we want both to run when the job finishes. To do so we just need to create the **MonitorSuite** in our `monitors.py` file:

monitors.py

class SpiderCloseMonitorSuite(MonitorSuite):

monitors = [ItemCountMonitor, ItemValidationMonitor,]

And then enable this **MonitorSuite** in our `settings.py` file:

settings.py

SPIDERMON_SPIDER_CLOSE_MONITORS = ('spidermon_demo.monitors.SpiderCloseMonitorSuite',)

MonitorSuite 2: Periodic Monitor

Setting up a periodic monitor to run every 5 seconds is just as easy. Simply create a new **MonitorSuite** and in this case we're only going to have it run the **ItemValidationMonitor** every 5 seconds:

monitors.py

class PeriodicMonitorSuite(MonitorSuite): monitors = [ItemValidationMonitor,]

And then enable it in our `settings.py` file, where we also specify how frequently we want it to run:

```
SPIDERMON_PERIODIC_MONITORS = {
    'spidermon_demo.monitors.PeriodicMonitorSuite': 5,  # time in
```

```
seconds
    }
```

With both of these **MonitorSuites** setup, Spidermon will automatically run these **Monitors** and add the results to your Scrapy logs and stats.

Create Our Actions

Having the results of these **Monitors** is good, but to make them really useful we want something to happen when a **MonitorSuite** has completed its tests.

The most common action is getting notified of a failed health check so for this example we're going to send a Slack notification.

First we need to install some libraries to be able to work with Slack:

pip install slack slackclient jinja2

Next we will need to enable Slack notifications in our **MonitorSuites** by importing `SendSlackMessageSpiderFinished` from Spidermon actions, and updating our **MonitorSuites** to use it.

monitors.py

from spidermon.contrib.actions.slack.notifiers import SendSlackMessageSpiderFinished

... Existing Monitors

Update Spider Close MonitorSuite

class SpiderCloseMonitorSuite(MonitorSuite):

monitors = [ItemCountMonitor, ItemValidationMonitor,]

monitors_failed_actions = [SendSlackMessageSpiderFinished,]

Update Periodic MonitorSuite

class PeriodicMonitorSuite(MonitorSuite): monitors = [ItemValidationMonitor,]

monitors_failed_actions = [SendSlackMessageSpiderFinished,]

Then add our Slack details to our `settings.py` file:

```
## settings.py
SPIDERMON_SLACK_SENDER_TOKEN = '<SLACK_SENDER_TOKEN>'
SPIDERMON_SLACK_SENDER_NAME = '<SLACK_SENDER_NAME>'
SPIDERMON_SLACK_RECIPIENTS = ['@yourself', '#yourprojectchannel']
```

Use this guide to create a Slack app and get your Slack credentials.

From here, anytime one of your Spidermon **MonitorSuites** fail, you will get a Slack notification.

The full code from this example is available on Github here.

More Scrapy Tutorials

That's it for how to use Spidermon to monitor your Scrapy spiders. If you would like to learn more about Scrapy, then be sure to check out The Scrapy Playbook.